Synopsis: AML Node16 Challenge

by
S M Ahasanul Karim

Task Description: This assignment tests a federated learning approach for classifying money laundering using advanced feature engineering and ensemble machine learning across 3 banks of varying sizes.

Synthetic Dataset: This task uses the dataset from AML-World, "NEW AND IMPROVED AML DATA" available in the GitHub repository *https://github.com/IBM/AML-Data* . From the Kaggle link, three different datasets are chosen to mimic different scales of banking and laundering activities

- HI-Large_Trans.csv(17.05 GB) - Large bank, high intensity of laundering from 2022-08-01 to 2023-01-12.
- LI-Medium_Trans.csv(2.98 GB) - Medium bank, low Intensity of laundering from 2022-09-01 to 2022-09-27
- HI-Small_Trans.csv(475.66 MB) - Small bank, high intensity of laundering from 2022-09-01 to 2022-09-18

The datasets are in a daily format and are distinct from one another without any kind of overlap. The publisher claims it is a better version than the AMLSim generator because it can only model 8 common patterns of money laundering solely on the layering phase, as shown in Figure 1, whereas this dataset takes care of the entire laundering cycle of sourcing, layering and integration, improving on the pioneering efforts of AMLSim.[1]

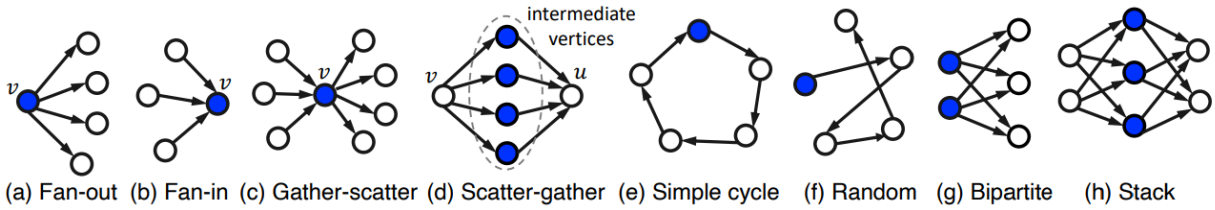

Figure1: Money Laundering Patterns in AML-sim.

The dataset builds around a multi-agent virtual world of banks, individuals, and companies with activities like buying items, and making bank transfers to get supplies, pay salaries, pay pensions etc with good and bad actors, where bad actors are doing things like smuggling, attempting to launder their illicit funds result in money-laundering transactions, plus other natura, laundering through criminal activities. The data produced is a set of transactions, as

[1] https://arxiv.org/pdf/2306.16424

illustrated in Figure 2, with each transaction labelled as laundering (illicit) or not, with columns “Timestamp”, “From Bank”, “Account”,
“To Bank”, “Account”, ”Amount Received”, “Receiving Currency”,” Amount Paid”, “Payment Currency”, “Payment Format”. There is also a patterns.txt file with details of all money laundering patterns and their corresponding cycle and timespan, from which it has been discovered that there is a coherent distribution of all kinds of money laundering patterns across all the datasets, with no major kind of laundering pattern. Regarding timespan, most of the patterns typically lie between a week, whereas for the large bank, some money laundering Gather-Scatter cycles can run up to two months.

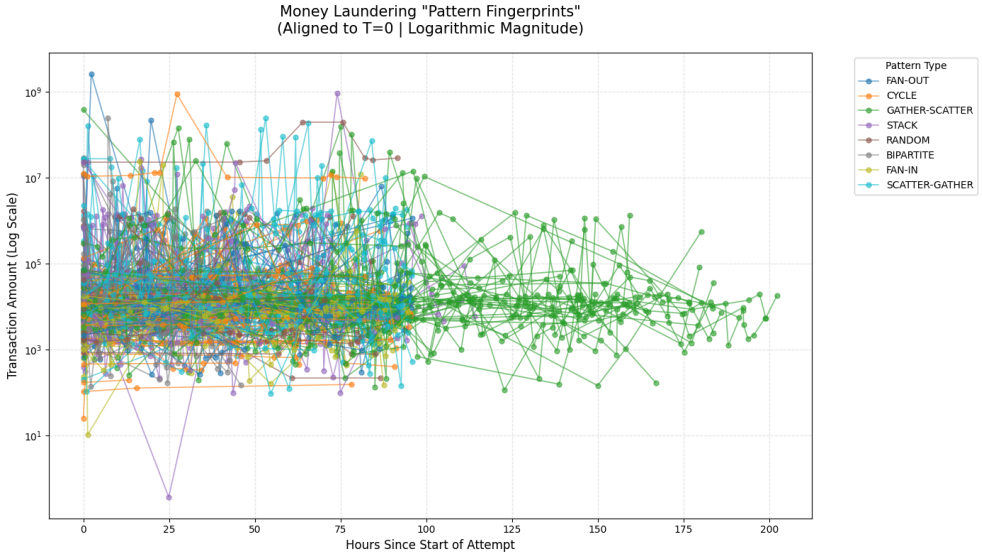

Figure 2: Typical Hourly Length of ML Patterns on Small Bank Data.

Feature Engineering: By identifying these trends, its evident that the problem with money laundering transaction data is that it requires temporal information to function properly; a single row in the existing data does not hold the previous information regarding that transaction. As these can be taken care of by GNN’s, for other models we need to include features that can have these informations. Therefore, several new features were added with these functions to enhance the prediction power of the model described below:

- Transaction_deltas: Calculates the time elapsed since the last transaction for both the sender and the receiver perspectives.
- Transaction_frequency: In hourly, weekly, biweekly, or monthly format.
- Cyclical_feature: Trigonometric estimation to discover hourly/daily transaction patterns.
- Off_hours_activity: For detecting late-night transactions.
- Network_topologies: To detect fan out and fan in within 7 days.
- Amount_ratios: To detect anomalies for large transactions with rolling avg, ratio and standard deviation.
- Balance_wash: To detect burner accounts emptying all their balance in a single transaction after receiving it immediately.

- Other discrepancy features: currency mismatch, forex trade, amount mismatch, etc.

The functions were written in a reusable format so that they can work across all the banks' data provided.

Again, the data shows a great amount of class imbalance(almost 0.1%), with the positive cases being too few in comparison to the negative cases. To preserve the integrity of the data, strategic balanced undersampling has been done rather than SMOTE. Because it can create synthetic transactions by interpolating between data points, which may create transactions that are mathematically impossible in the real banking world. In the applied ratio-based approach, 100% of the minority classes were kept, and a strategic slice of the majority was added in a 1:2 ratio.

Also, the sampling has done in a way so that when split into 80:20:20 train, test & validation sets, the data can have a coherent distribution of negative and positive cases in all of them. The splitting can not be random in a banking data set, as it has to maintain its temporal integrity, and we cannot go back in time to predict old cases in reality. Therefore, for the big bank case, only negative entries in between positive entries were sampled so that the distribution is not of deceiving standards.

Bank_type	Train set	Validation set	Test set
HI-Small	114110 positive 291872 negative	48524 positive 86804 negative	62912 positive 72416 negative
LI-Medium	8730 positive 20143 negative	3709 positive, 5916 negative	3602 positive 6023 negative
HI-Large	2565 positive 6753 negative	1045 positive, 2061 negative	1567 positive 1540 negative

Table 1. Class Distribution in train, test and validation sets across all banks.

Model Selection: "Light Gradient Boosting Method" has been selected as the model for classifying money laundering in this project because it is not sensitive to the curse of dimensionality. Therefore, its performance is not affected by a huge number of features. Also, It can inherently handle categorical features without the need for encodings, which can create big issues and make the training process inefficient. Additionally, It can handle class imbalance inherently with parameters. On top of that, it is computationally less expensive than other popular ensemble models in comparison. Also, it is more explainable than GNNs or Neural Nets.

Also, **Precision-Recall Area Under Curve** has been used as a primary metric for hyperparameter tuning and evaluation, as it estimates the tradeoff between false negatives and false positives. The hyperparameters were trained on the big bank data, and then the best

model was used to train all the bank data locally. Then there has been a federated round where the model was trained across all the banks and then evaluated with their difference in metrics.

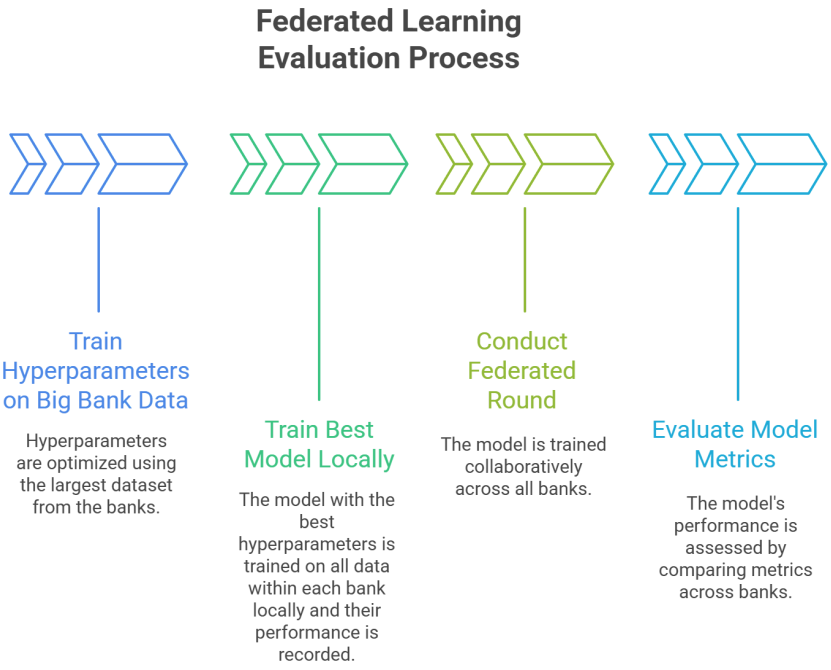

Fig 3. Federated Learning Experiment Design

After training this model locally and in a federated setup, the following results were achieved on the test sets across all three banks, with the better-performing one underlined:

Bank	Model Type	PR-AUC	ROC-AUC	F1-Score	Precision	Recall	Accuracy
Large Bank	Local	0.9779	0.9814	0.9081	0.8475	0.9781	0.908
	Federated	0.9732	0.978	0.9151	0.9125	0.9177	0.9208
Medium Bank	Local	0.9119	0.9525	0.8597	0.7767	0.9625	0.8824
	Federated	0.917	0.9521	0.8048	0.8587	0.7574	0.8625
Small Bank	Local	0.9756	0.9766	0.916	0.9037	0.9285	0.9141
	Federated	0.9769	0.9774	0.9151	0.9151	0.9151	0.9144

From the results its evident that the performance has improved for smaller banks, whereas for larger banks it's almost the same. Also, as much as the federated learning takes place, the precision increases, and the recall decreases, which might help prevent overfitting in certain conditions. It will be interesting to further test with more data/banks for further interesting insights.